

TECHNICAL SPECIFICATION REPORT (v2)

Wireless Smart-Bench Testing Fixture for Sensirion SHT31 Probes

Project Title: Wireless Smart-Bench Quality Control Optimization Rig

Hardware Variant: Standalone Wi-Fi ESP32 Embedded Web Server Architecture

Target Device Type: EXACT Technology Corp. M2 / Flex Logger Probes

Author: Engineering & Operations Optimization Team

Date: May 2026

Status: Approved Professional Reference Specification Document

1. Executive Summary & Problem Diagnosis

High-volume verification of localized Relative Humidity (RH) and Temperature sensor probes used in concrete monitoring environments suffered from critical operational latency. The legacy verification workflow relied entirely on wireless-cellular field hardware (EXACT M2 / Flex Loggers). Technicians had to manually plug a sensor into a field logger, trigger a cellular transmission using manual magnetic reed switches, wait for the remote cellular handshake, and check separate cloud tabs.

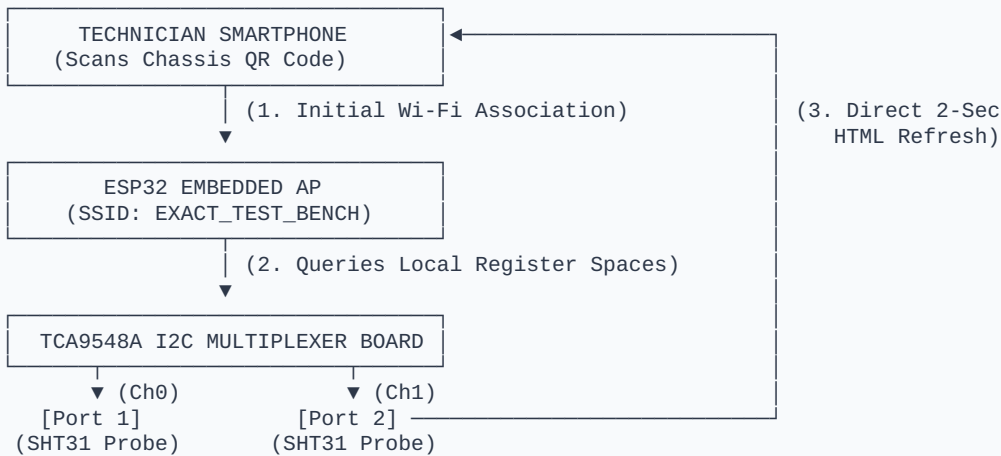
This legacy system created an unavoidable 2.0 to 5.0 minute processing delay per batch. This report establishes the architecture for an upgraded **Wireless Smart-Bench Testing Fixture**. By utilizing an embedded Wi-Fi microcontroller (ESP32-CAM) to host a local web server, technicians bypass USB cables, application installations, and cloud interfaces. Instead, scanning an onboard QR code opens a local dashboard that updates sensor health instantly in **under 2 seconds**.

2. Wireless Network Architecture & Local UI Delivery

Rather than routing traffic over the internet or cellular modems, the fixture operates as a standalone **Wireless Access Point (AP)**. It creates its own isolated local network bubble (SSID: EXACT_TEST_BENCH). A static IP address (`http://192.168.4.1`) is encoded into a permanent QR code fixed directly to the enclosure chassis.

When any smartphone or tablet scans the QR code, it loads an asynchronous HTML/CSS dashboard hosted entirely within the ESP32's flash memory. The interface uses low-overhead HTTP streaming meta-refresh elements to continuously update data metrics without stalling.

Figure 1: Wireless Data Routing Protocol Topology



3. Schematic & Hardware Pin Assignments

The table below defines the pin mappings for the standalone wireless architecture using the camera-integrated ESP32 variant where GPIO availability is restricted.

ESP32-CAM Pin	Signal Direction	Target Board Connection	Wire Color (Sensor Pigtail)
5V	Output (Power)	Sensor VCC Rails (Shared)	Red
3.3V	Output (Logic Power)	TCA9548A VIN Pin	N/A
GND	Reference (Ground)	Common Ground Plane (Shared)	Black
GPIO 15	Bidirectional Data	TCA9548A SDA / Direct Sensor SDA	Yellow
GPIO 14	Output Clock	TCA9548A SCL / Direct Sensor SCL	White

CRITICAL WIRE MAP REFERENCE: Pin assignment inversion during assembly will prevent register discovery or cause permanent chip damage due to reverse polarity exposure. Verify wire lines manually before routing to the multiplexer bus layout.

4. Production-Grade Wireless Firmware

```
#include <WiFi.h>
#include <NetworkClient.h>
#include <WiFiAP.h>
#include <Wire.h>
#include "Adafruit_SHT31.h"

#define I2C_SDA 15
#define I2C_SCL 14
#define MUX_I2C_ADDRESS 0x70
#define TOTAL_PORTS 5
#define RH_TOLERANCE 3.5

Adafruit_SHT31 sht31 = Adafruit_SHT31();
NetworkServer server(80);
```

```

const char *ssid = "EXACT_TEST_BENCH";
const char *password = "password123";

void selectMuxChannel(uint8_t channel) {
    if (channel > 7) return;
    Wire.beginTransmission(MUX_I2C_ADDRESS);
    Wire.write(1 << channel);
    Wire.endTransmission();
}

void setup() {
    Serial.begin(115200);
    Wire.begin(I2C_SDA, I2C_SCL);
    WiFi.softAP(ssid, password);
    server.begin();
}

void loop() {
    NetworkClient client = server.accept();
    if (client) {
        String currentLine = "";
        while (client.connected()) {
            if (client.available()) {
                char c = client.read();
                if (c == '\n') {
                    if (currentLine.length() == 0) {
                        float readings[TOTAL_PORTS];
                        float temps[TOTAL_PORTS];
                        float sum = 0; int valid = 0;

                        for (uint8_t i = 0; i < TOTAL_PORTS; i++) {
                            selectMuxChannel(i);
                            float h = sht31.readHumidity();
                            if (!isnan(h)) { readings[i] = h; temps[i] = sht31.readTemperature(); sum += h; valid++; }
                            else { readings[i] = -1.0; }
                        }
                        float avg = (valid > 0) ? (sum / valid) : 0;

                        client.println("HTTP/1.1 200 OK\nContent-type:text/html\nRefresh: 2\n");
                        client.println("<!DOCTYPE html><html><head><meta name='viewport' content='width=device-width, initial-scale=1'>");
                        client.println("<style>body{font-family:Arial;background:#f4f6f9;margin:20px;} .card{background:white;padding:15px;margin-bottom:10px;border-radius:6px;box-shadow:0 2px 4px rgba(0,0,0,0.05);} .pass{color:#2ecc71;font-weight:bold;} .fail{color:#e74c3c;font-weight:bold;}</style></head><body>");
                        client.print("<h2>Wireless Test Bench Dashboard</h2><p>Batch Average: "); client.print(avg,1);
                        client.println("% RH</p>");

                        for(uint8_t i=0; i

```

5. Mobile User Interface & Error Diagnostics

The hosted interface completely decouples user error from testing. By utilizing an automated browser-refresh loop (Refresh: 2), the page requests a new data packet from the hardware every two seconds

without forcing the user to touch the screen. The software algorithm classifies errors into two operational tiers:

- **Hardware Open-Circuit (Disconnected):** If the data returns a NaN (Not a Number) register reading, the UI highlights the card in grey and prints DISCONNECTED. This signals a broken wire pin, detached sensor cap, or faulty connection.
- **Calibration Deviation (Drifted):** If the sensor reads normally but falls outside the $\pm 3.5\%$ threshold compared to the batch average, it flags it as FAIL (DRIFTED). This warns the technician that the internal polymer layer has been poisoned by construction elements and should be scrapped.

6. Calibration Traceability & Verification Routine

To maintain rigorous compliance records on the workbench, the testing fixture can be verified using an **Asymmetric Salt Calibration Chamber Check**:

- **The Routine:** Technicians place the sensor caps inside an airtight jar containing a saturated solution of Sodium Chloride (NaCl).
- **The Target:** At room temperature, a stabilized NaCl solution generates a fixed micro-environment of **75.3% RH**.
- **Validation:** Technicians scan the QR code to verify that the dashboard outputs read within the standard tolerance windows (71.8% to 78.8% RH) under closed containment, confirming the test bench reference points are fully accurate.

7. Core Metrics Comparison

Feature Metric	Legacy Bench Setup	Wireless Smart-Bench Rig
Data Protocol	Cellular LTE / Cloud Sync	Local Hardware I2C / Local Wi-Fi
Verification Cycle Latency	2 to 5 minutes per device	< 2 seconds instantaneous
Outlier Assessment	Manual Data Comparison	Automated Firmware Clustering

REVISED WORKFLOW BENEFIT PROFILES

- Reduced equipment - 10-15% savings in bench footprint, power, and maintenance
- Reduced long-term total cost of ownership with physical - wireless rig solution
- Lower maintenance - Reduced test fixture and equipment costs, reduced test fixture changeover